

Examen Práctico Unidad 1

RegistroDistribuido — Proceso de implementación

Asignatura: Aplicaciones Distribuidas (ISR-701)

Estudiante:Isaias Abraham Urbina Romero

Unidad: 1 — Generalidades de los Sistemas Distribuidos

Universidad: Universidad Técnica Estatal de Quevedo

Período: Regular 2025-2026 SPA

Enlace del repositorio: <https://github.com/IsaiasUrb/Aplicaciones-Distribuidas-IU.git>

Estado real del proyecto

Por limitación de tiempo durante el examen, se logró implementar completamente las siguientes partes:

Parte	Descripción	Estado
A	Comunicación TCP cliente–nodo	Implementado
B	Relojes lógicos de Lamport	Implementado
C	Tolerancia a fallos (heartbeat)	No implementado
D	Elección de coordinador (Bully)	No implementado
E	Seguridad (autenticación por token)	No implementado

Las clases correspondientes a las partes C, D y E existen en el proyecto pero están vacías. Se documentan a continuación los procesos y códigos de lo que sí se implementó, y se explica el diseño pensado para las partes pendientes.

Configuración del proyecto — pom.xml

El proyecto usa Maven como gestor de dependencias. El artefacto se llama `RegistroDistribuido` bajo el grupo `ec.edu.uteq.distribuidas`.

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>ec.edu.uteq.distribuidas</groupId>
  <artifactId>RegistroDistribuido</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>

  <name>RegistroDistribuido</name>
  <url>http://maven.apache.org</url>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>

```

Parte A — Comunicación entre procesos (TCP)

Proceso realizado

Se implementó la comunicación cliente–nodo mediante sockets TCP. El servidor es multihilo: usa un `ExecutorService` con pool fijo de 10 hilos para atender múltiples clientes concurrentemente. La delimitación de mensajes se hace por líneas de texto (`println` / `readLine`), usando `\n` como separador, lo que evita que dos mensajes enviados seguidos se solapen o trunquen en el receptor.

El servidor procesa tres comandos: `HORA`, `ECO <texto>` y `SALIR`.

Código — ServidorTCP.java

```

package ec.edu.uteq.distribuidas.nodo.Comunicacion;

import java.io.*;
import java.net.*;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

/**
 * Servidor TCP multihilo con pool de hilos fijo.
 * Escucha en el puerto 9008 y atiende comandos:
 * HORA -> retorna la hora actual del servidor
 * ECO -> retorna el mensaje prefijado con ECO:
 * SALIR -> cierra la conexión del cliente
 */
public class ServidorTCP {

    private static final int PUERTO = 9008;
    private static final int MAX_HILOS = 10;

    private static final AtomicInteger contadorClientes =
        new AtomicInteger(0);

    public static void main(String[] args) {

        ExecutorService pool =
            Executors.newFixedThreadPool(MAX_HILOS);

        try (ServerSocket servidor =
            new ServerSocket(PUERTO)) {

            System.out.println("[ " + timestamp()
                + " ] Servidor listo en: "
                + PUERTO);

            while (true) {

                Socket cliente = servidor.accept();

                int id =
                    contadorClientes.incrementAndGet();

                System.out.println("[ " + timestamp()
                    + " ] Cliente #"
                    + id
                    + " conectado desde "
                    + cliente.getInetAddress());

                pool.submit(
                    new ManejadorCliente(cliente, id)
                );

            }

        } catch (IOException e) {

            System.err.println(
                "Error en servidor: "
                + e.getMessage()
            );

        }
    }
}

```

```

    }
}

public static String timestamp() {

    return LocalDateTime.now()
        .format(
            DateTimeFormatter.ofPattern(
                "HH:mm:ss.SSS"
            )
        );
}

static class ManejadorCliente implements Runnable {

    private final Socket socket;
    private final int idCliente;

    ManejadorCliente(Socket socket, int idCliente) {
        this.socket = socket;
        this.idCliente = idCliente;
    }

    @Override
    public void run() {

        try {
            BufferedReader entrada =
                new BufferedReader(
                    new InputStreamReader(
                        socket.getInputStream()
                    )
                );

            PrintWriter salida =
                new PrintWriter(
                    socket.getOutputStream(),
                    true
                );

        } {

            String linea;

            while ((linea = entrada.readLine()) != null) {

                System.out.println(
                    "[" + ServidorTCP.timestamp()
                    + "] Cliente#"
                    + idCliente
                    + " > "
                    + linea
                );

                if (linea.equalsIgnoreCase("HORA")) {

                    salida.println(
                        "HORA_ACTUAL: "
                        + ServidorTCP.timestamp()
                    );
                }
            }
        }
    }
}

```

```

        } else if (linea.toUpperCase().startsWith("ECO")) {

            String mensaje =
                linea.length() > 4
                ? linea.substring(4)
                : "(vacío)";

            salida.println(
                "ECO [#"
                + idCliente
                + "]: "
                + mensaje
            );

        } else if (linea.equalsIgnoreCase("SALIR")) {

            salida.println(
                "ADIOS: hasta luego cliente #"
                + idCliente
            );

            System.out.println(
                "[" + ServidorTCP.timestamp()
                + "] Cliente #"
                + idCliente
                + " desconectado."
            );

            break;

        } else {

            salida.println(
                "ERROR: comando desconocido -> "
                + linea
            );

        }

    } catch (IOException e) {

        System.err.println(
            "Error cliente #"
            + idCliente
            + ": "
            + e.getMessage()
        );

    }

}

}

}

```

Código — ClienteTCP.java

```

package ec.edu.uteq.distribuidas.nodo.Comunicacion;

import java.io.*;
import java.net.*;
import java.util.Scanner;

/**
 * Cliente TCP interactivo.
 * Envía comandos al servidor y muestra la respuesta.
 */
public class ClienteTCP {

    private static final String HOST = "localhost";
    private static final int PUERTO = 9008;

    public static void main(String[] args)
        throws IOException {

        try {

            Socket socket =
                new Socket(HOST, PUERTO);

            BufferedReader entrada =
                new BufferedReader(
                    new InputStreamReader(
                        socket.getInputStream()
                    )
                );

            PrintWriter salida =
                new PrintWriter(
                    socket.getOutputStream(),
                    true
                );

            Scanner teclado =
                new Scanner(System.in)
        ) {

            System.out.println(
                "Conectado a " + HOST + ":" + PUERTO
            );

            System.out.println("Comandos disponibles:");
            System.out.println("HORA | ECO <texto> | SALIR");
            System.out.println("-----");

            String comando;

            while (true) {

                System.out.print("> ");

                comando = teclado.nextLine().trim();

                if (comando.isEmpty()) {
                    continue;
                }

                salida.println(comando);

```

```

        String respuesta = entrada.readLine();

        System.out.println("Servidor: " + respuesta);

        if (comando.equalsIgnoreCase("SALIR")) {
            break;
        }
    }

    System.out.println("Conexión cerrada.");
}
}

```

Clases pendientes de esta parte

`Mensajes.java` y `Operacion.java` quedaron vacías. La intención era que `Mensajes` encapsulara el protocolo de framing y `Operacion` modelara los datos de cada registro (id, descripción, timestamp).

Parte B — Relojes lógicos de Lamport ■

Proceso realizado

Se implementó el reloj lógico de Lamport en la clase `Lamport`. Todos los métodos son `synchronized` para garantizar que múltiples hilos del servidor accedan al reloj sin condiciones de carrera.

- **`tick()`** — Evento local: incrementa el contador y retorna el nuevo valor. Se debía llamar antes de enviar un mensaje o al registrar una operación.
- **`update(int recibido)`** — Evento de recepción: aplica $\max(\text{local}, \text{recibido}) + 1$, garantizando que la recepción sea causalmente posterior al envío.
- **`getTiempo()`** — Consulta sin modificar.

La integración de este reloj con `ServidorTCP` no llegó a completarse por falta de tiempo; la clase existe y compila correctamente pero no se instancia desde el servidor.

Código — Lamport.java

```

package ec.edu.uteq.distribuidas.nodo.Relojes;

public class Lamport {

    private int tiempo = 0;

    /**
     * Evento local: incrementa el reloj.
     */
    public synchronized int tick() {
        return ++tiempo;
    }

    /**
     * Evento de recepción: max(local, recibido) + 1.
     */
    public synchronized int update(int recibido) {
        tiempo = Math.max(tiempo, recibido) + 1;
        return tiempo;
    }

    /**
     * Consulta el valor actual sin modificarlo.
     */
    public synchronized int getTiempo() {
        return tiempo;
    }
}

```

Partes C, D y E — No implementadas

Las clases `NodoApp.java`, `BullyAlgorit.java` y `registroEventLog.java` existen en el proyecto pero están vacías. No se alcanzó a implementar el heartbeat, la elección de coordinador ni la autenticación por token. Se reconoce que estas partes son funcionales para el cumplimiento completo del examen.

Preguntas de análisis

1. Teorema CAP

La implementación actual (solo `ServidorTCP`) privilegia la **Disponibilidad (A)** sobre la **Consistencia (C)**, ubicándose en la categoría **AP**. El servidor sigue atendiendo clientes aunque no haya coordinación con otros nodos, lo que puede producir estados divergentes entre réplicas. No se implementó quórum ni bloqueo por consenso, así que ante una partición de red, cada nodo operaría de forma independiente sin garantizar que todos vean el mismo estado.

2. Falacias de la computación distribuida

"La red es confiable": Al usar `readLine()`, una desconexión abrupta a mitad de un mensaje genera una `IOException`. Por eso el `ManejadorCliente` la captura y la aísla al hilo del cliente sin tumbar el servidor completo.

"La latencia es cero": Si se hubiera implementado el heartbeat, el timeout debía calibrarse cuidadosamente: demasiado corto genera falsos positivos (marcar caído un nodo lento), demasiado largo retrasa la detección real de una falla.

"La red es segura": Los tokens de autenticación viajarían en texto plano sobre TCP sin TLS. En el prototipo se asume red de laboratorio; en producción se requeriría cifrado.

3. SLA propuesto

SLA: 99.5% de disponibilidad

Métrica	Valor
Inactividad permitida	0.5%
Minutos al año ($365 \times 24 \times 60$)	525,600 min
Inactividad anual máxima	2,628 min
Equivalente	≈ 43.8 h/año
Por mes	≈ 3.65 horas

Con dos nodos activos (uno caído) el sistema seguiría operando. El tiempo de detección del fallo más reelección de coordinador suma aproximadamente 15 segundos por incidente, lo que es compatible con este SLA.

4. Bully vs Raft

Ventajas de Raft:

- Garantiza consistencia fuerte: las operaciones solo se confirman una vez que la mayoría las ha registrado.
- Elimina el split-brain: nunca puede haber dos líderes activos al mismo tiempo.
- Incluye replicación de log integrada; con Bully habría que implementarla aparte.

Costos de Raft:

- Mucho más complejo de implementar correctamente: requiere términos, votos, AppendEntries y recuperación de log divergente.
- Mayor latencia por operación: cada escritura espera quórum (2/3 nodos).
- Ventana de no disponibilidad durante elecciones (150–300 ms típicamente).

Para un prototipo académico Bully es suficiente para demostrar la elección de líder. Raft sería la opción correcta en un sistema productivo donde la consistencia importa.



Inicio (/) /

[← Atrás \(/alu_documentos?action=resumencuestionario&id=OOPPPQQRRRSSRTQOQL\)](#)



Revisión de intento

APLICACIONES DISTRIBUIDAS - SOFT-R - A - [7MO NIVEL] | CORTE 1

Estudiante	URBINA ROMERO ISAIAS ABRAHAM
Comenzado el	Martes, 2 de Junio de 2026, 07:56
Intento	1 / 1
Estado	Finalizado
Finalizado en	Martes, 2 de Junio de 2026, 08:06
Tiempo empleado	0:10:00
Calificación obtenida	5.50 / 20.00
Calificación final	2.75 / 10.00

PREGUNTA

1

Finalizado

1.00 / 1.00

Un sistema cifra todos los datos en tránsito con TLS, pero almacena las contraseñas en texto plano en la base de datos. Un atacante que compromete la base de datos obtiene todas las contraseñas. ¿Qué principio de seguridad se violó?

Seleccione una alternativa:

- ☐ a. El no repudio o denegación de servicios (DOS), ya que el atacante cambió las contraseñas actuales.
- ☐ b. La integridad de los datos en tránsito
- ☐ c. La seguridad en reposo; el cifrado en tránsito no protege los datos almacenados.
- ☒ d. La confidencialidad en reposo; el cifrado en tránsito no protege los datos almacenados. ✓

La respuesta correcta es: La confidencialidad en reposo; el cifrado en tránsito no protege los datos almacenados.

PREGUNTA

2

Finalizado

0.00 / 1.00

Un equipo necesita invocar métodos sobre objetos remotos entre dos máquinas virtuales de Java, pasando objetos como parámetros y manejando automáticamente la serialización. ¿Qué mecanismo es el más apropiado y por qué?

Seleccione una alternativa:

- ☐ a. Invocación remota de métodos, porque está diseñada para invocar métodos sobre objetos remotos en el ecosistema Java mediante stubs y serialización.
- ☒ b. TCP, porque garantiza la entrega ordenada y fiable de los objetos ✖
- ☐ c. Sockets TCP crudos, porque ofrecen el mayor control de bajo nivel.
- ☐ d. Llamada a procedimiento remoto clásica estilo C, porque soporta orientación a objetos de forma nativa.

La respuesta correcta es: Invocación remota de métodos, porque está diseñada para invocar métodos sobre objetos remotos en el ecosistema Java mediante stubs y serialización.

PREGUNTA

3

Finalizado

0.00 / 1.00

Se compara un sistema centralizado con uno distribuido bajo la misma carga. ¿Cuál es una desventaja inherente al distribuido que el centralizado no presenta?

Seleccione una alternativa:

- ☐ a. La ausencia total de concurrencia.
- ☒ b. La imposibilidad de escalar horizontalmente. ✖
- ☐ c. La existencia de un único punto de fallo total del sistema.
- ☐ d. La posibilidad de fallo parcial, donde unos nodos fallan y otros siguen operando, generando estados intermedios inconsistentes.

La respuesta correcta es: La posibilidad de fallo parcial, donde unos nodos fallan y otros siguen operando, generando estados intermedios inconsistentes.

PREGUNTA

4

Finalizado

1.00 / 1.00

Un equipo intenta mejorar el rendimiento de una aplicación que envía miles de mensajes muy pequeños entre dos nodos. Duplica el ancho de banda del enlace, pero el rendimiento prácticamente no cambia. ¿Qué falacia de la computación distribuida explica este resultado?

Seleccione una alternativa:

- ☐ a. "El transporte es gratuito".
- ☐ b. "La red es confiable"
- ☐ c. "El ancho de banda es infinito".
- ☒ d. "La latencia es cero". ✓

La respuesta correcta es: "La latencia es cero".

PREGUNTA

5

Finalizado

1.00 / 1.00

En el protocolo de 2PC, ¿cuál es su principal debilidad y por qué se produce?

Seleccione una alternativa:

- ☐ a. Que es no determinista en su resultado, en vista de que la comunicación entre las 2PC puede romperse.
- ☐ b. Que requiere relojes físicos sincronizados entre los participantes y que no garantiza la atomicidad de la transacción
- ☒ c. Que es bloqueante: si el coordinador falla tras la fase de votación, los participantes que votaron "sí" pueden quedar bloqueados indefinidamente esperando la decisión final. ✓
- ☐ d. Que es desbloqueante: si el coordinador falla tras la fase de votación, los participantes que votaron "sí" quedan libres del bloqueo indefinidamente esperando la decisión final.

La respuesta correcta es: Que es bloqueante: si el coordinador falla tras la fase de votación, los participantes que votaron "sí" pueden quedar bloqueados indefinidamente esperando la decisión final.

PREGUNTA

6

Finalizado

0.00 / 1.00

Sockets TCP

Relacione la opción correcta:

Middleware orientado a mensajes (colas)

Comunicación Asíncrona Y Desacoplada, Donde Emisor Y Receptor No Necesitan Coincidir En El Tiempo.



Sockets UDP

Sincronización De Relojes Físicos De Los Nodos Mediante Señal Satelital.



Invocación remota de métodos (RMI)

Invocación De Métodos Sobre Objetos Remotos Con Paso De Objetos Serializados Mediante Stubs.



La respuesta correcta es: Invocación remota de métodos (RMI) → Datagramas sin garantía de entrega ni orden, a cambio de menor latencia. , Middleware orientado a mensajes (colas) → Invocación de métodos sobre objetos remotos con paso de objetos serializados mediante stubs. , Sockets UDP → Flujo de bytes fiable y ordenado que no preserva los límites entre mensajes.

PREGUNTA

7

Finalizado

0.25 / 1.00

Relaciones: Protocolo o algoritmo de coordinación y su propósito clave

Relacione la opción correcta:

Algoritmo Bully

Logra Acuerdo Sobre Un Único Valor Mediante Quórum De Mayoría, Tolerando Caídas De Nodos.



Relojes de Lamport

Ordena Causalmente Los Eventos Sin Dependier De Un Reloj Físico Global.



Consenso Raft / Paxos

Divide Una Tabla En Subconjuntos Disjuntos De Filas Distribuidos Entre Nodos.



Commit en dos fases (2PC)

Cifra El Tráfico Entre Nodos Para Impedir Su Lectura Por Terceros.



La respuesta correcta es: Commit en dos fases (2PC) → Garantiza que una transacción se confirme en todos los nodos o en ninguno, aunque puede quedar bloqueado si el coordinador cae. , Algoritmo Bully → Elige como coordinador al nodo activo de mayor identificador tras detectar una caída. , Consenso Raft / Paxos → Logra acuerdo sobre un único valor mediante quórum de mayoría, tolerando caídas de nodos. , Relojes de Lamport → Ordena causalmente los eventos sin depender de un reloj físico global.

PREGUNTA

8

Finalizado

1.00 / 1.00

Un desarrollador usa sockets TCP para enviar por separado los mensajes "HOLA" y "MUNDO", pero en el receptor a veces llegan juntos como "HOLAMUNDO" en una sola lectura. ¿Cuál es la explicación correcta?

Seleccione una alternativa:

- ☒ a. TCP es un protocolo orientado a flujo (stream) que no preserva los límites de mensaje, por lo que la aplicación debe delimitarlos explícitamente ✓
- ☐ b. TCP perdió y luego retransmitió un mensaje.
- ☐ c. El sistema cambió automáticamente a UDP.
- ☐ d. Es un error físico de la tarjeta de red

La respuesta correcta es: TCP es un protocolo orientado a flujo (stream) que no preserva los límites de mensaje, por lo que la aplicación debe delimitarlos explícitamente

PREGUNTA

9

Finalizado

1.00 / 1.00

¿Cuál enunciado distingue correctamente la "tolerancia a fallos" de la "prevención de fallos"?

Seleccione una alternativa:

- ☐ a. La prevención de fallos solo es aplicable al hardware, mientras que la tolerancia a fallos es tanto al software como al hardware.
- ☐ b. Son términos sinónimos.
- ☒ c. La tolerancia a fallos busca que el sistema siga operando correctamente a pesar de que los fallos ocurran, mientras que la prevención intenta evitar que ocurran. ✓
- ☐ d. La tolerancia a fallos elimina por completo todos los fallos posibles, mientras que la prevención pone en alerta al administrador sobre posibles fallos.

La respuesta correcta es: La tolerancia a fallos busca que el sistema siga operando correctamente a pesar de que los fallos ocurran, mientras que la prevención intenta evitar que ocurran.

PREGUNTA

10

Finalizado

0.25 / 1.00

Relaciones: Garantía o comportamiento observado y su modelo/concepto

Relacione la opción correcta:

Tras una escritura, algunas lecturas devuelven valores antiguos durante un lapso, pero todas terminan coincidiendo si cesan las escrituras.

Tolerancia A Fallos Bizantinos



Un proceso siempre ve sus propias escrituras en orden, aunque otros procesos las vean con retraso.

"Lee Tus Propias Escrituras" (Read-Your-Writes)



El sistema confirma una operación únicamente cuando más de la mitad de los nodos la han registrado.

Consistencia Fuerte (Linealizable)



Tras escribir un dato, cualquier lectura posterior en cualquier nodo devuelve ese valor de inmediato.

Reloj Lógico De Lamport



La respuesta correcta es: Tras escribir un dato, cualquier lectura posterior en cualquier nodo devuelve ese valor de inmediato. → Consistencia fuerte (linealizable) , Un proceso siempre ve sus propias escrituras en orden, aunque otros procesos las vean con retraso. → "Lee tus propias escrituras" (read-your-writes) , Tras una escritura, algunas lecturas devuelven valores antiguos durante un lapso, pero todas terminan coincidiendo si cesan las escrituras. → Consistencia eventual , El sistema confirma una operación únicamente cuando más de la mitad de los nodos la han registrado. → Quórum de mayoría

PREGUNTA

11

Finalizado

0.00 / 1.00

Una aplicación en Windows escrita en C# y otra en Linux escrita en Java deben intercambiar mensajes sin que ninguna conozca el sistema operativo ni el lenguaje de la otra. ¿Qué propiedad del middleware lo hace posible y cuál es su costo principal?

Seleccione una alternativa:

- ☐ a. Elimina por completo la latencia de red; el costo es el ancho de banda que se cree que es ilimitado por eso no se toma en cuenta su afectación.
- ☒ b. Convierte el sistema distribuido en uno centralizado; el costo es la escalabilidad, dejando al sistema sin las posibilidades de crecer. ✖
- ☐ c. Garantiza que el sistema nunca fallará; el costo es el almacenamiento, que en nuestros días eso no es una limitación.
- ☐ d. Provee transparencia de acceso ocultando las diferencias de representación de datos; el costo es la sobrecarga de serialización y deserialización.

La respuesta correcta es: Provee transparencia de acceso ocultando las diferencias de representación de datos; el costo es la sobrecarga de serialización y deserialización.

PREGUNTA

12

Finalizado

0.00 / 1.00

¿Por qué los algoritmos de consenso como Paxos y Raft exigen la participación de una mayoría (quórum) de nodos para tomar decisiones, en lugar de uno solo o de la totalidad?

Seleccione una alternativa:

- ☐ a. Porque la mayoría reduce el ancho de banda total consumido, además porque un solo nodo es siempre suficiente para alcanzar el consenso
- ☐ b. Porque exigir a todos los nodos impediría progresar ante cualquier fallo, mientras que la mayoría garantiza que dos quórumes cualesquiera siempre se solapan en al menos un nodo, evitando decisiones contradictorias.
- ☒ c. Porque la falta de exigencia universal (a todos los nodos) impediría progresar ante cualquier fallo, mientras que la mayoría garantiza que dos quórumes cualesquiera siempre se solapan en al menos un nodo, evitando decisiones contradictorias. ✖
- ☐ d. Porque la mayoría siempre resulta más rápida que un solo nodo.

La respuesta correcta es: Porque exigir a todos los nodos impediría progresar ante cualquier fallo, mientras que la mayoría garantiza que dos quórumes cualesquiera siempre se solapan en al menos un nodo, evitando decisiones contradictorias.

PREGUNTA

13

Finalizado

0.00 / 1.00

Un acuerdo de nivel de servicio (SLA) promete una disponibilidad anual del 99,9 %. ¿Aproximadamente cuánto tiempo de inactividad permite al año, y qué implicaría elevarlo al 99,99 %?

Seleccione una alternativa:

- ☐ a. ≈ 1 minuto al año; el 99,99 % lo duplicaría.
- ☒ b. $\approx 3,65$ días al año; el 99,99 % no cambiaría nada. ✖
- ☐ c. ≈ 88 horas al año; el 99,99 % lo empeoraría.
- ☐ d. $\approx 8,76$ horas al año; al 99,99 % se reduciría a $\approx 52,6$ minutos al año.

La respuesta correcta es: $\approx 8,76$ horas al año; al 99,99 % se reduciría a $\approx 52,6$ minutos al año.

PREGUNTA

14

Finalizado

0.00 / 1.00

En el algoritmo Bully (del matón), un nodo con identificador 3 detecta la caída del coordinador y lanza una elección enviando mensajes a los nodos de identificador mayor (4, 5 y 6). Los nodos 5 y 6 están caídos, pero el nodo 4 responde. ¿Quién será el coordinador final?

Seleccione una alternativa:

- ☐ a. El nodo 6, porque es el de mayor identificador entre los nodos vivos.
- ☐ b. El nodo 4, porque es el de mayor identificador entre los nodos vivos.
- ☒ c. El nodo 3, porque fue quien inició la elección. ✖
- ☐ d. Ninguno, porque existen nodos caídos durante la elección

La respuesta correcta es: El nodo 4, porque es el de mayor identificador entre los nodos vivos.

PREGUNTA

15

Finalizado

0.00 / 1.00

Dos centros de datos de un banco mantienen copias sincronizadas de las mismas cuentas. Una falla en el enlace de fibra deja a ambos centros sin comunicación entre sí, aunque cada uno sigue funcionando por separado y recibiendo clientes. Ante esta situación, el sistema decide bloquear todas las operaciones de retiro hasta que se restablezca el enlace, prefiriendo que ningún cliente pueda operar antes que arriesgarse a que la misma cuenta termine reflejando saldos distintos en cada centro. Según el teorema CAP, ¿qué par de propiedades prioriza esta decisión?

Seleccione una alternativa:

- ☒ a. Disponibilidad y tolerancia a particiones (AP). ✖
- ☐ b. Las tres propiedades simultáneamente
- ☐ c. Consistencia y disponibilidad (CA)
- ☐ d. Consistencia y tolerancia a particiones (CP)

La respuesta correcta es: Consistencia y tolerancia a particiones (CP)

PREGUNTA

16

Sin contestar

0.00 / 1.00

Un sistema replicado requiere un quórum de mayoría (más de la mitad de los nodos) para confirmar una operación y mantener la consistencia. Con 5 réplicas, ¿cuántos fallos por caída simultáneos puede tolerar sin perder la capacidad de formar quórum?

Seleccione una alternativa:

- ☐ a. 2, porque el quórum de mayoría es 3 y se necesitan al menos 3 réplicas vivas
- ☐ b. 4, porque basta con que una réplica siga viva.
- ☐ c. 3, porque con 2 réplicas vivas todavía hay mayoría.
- ☐ d. 0, porque cualquier caída rompe el quórum.

La respuesta correcta es: 2, porque el quórum de mayoría es 3 y se necesitan al menos 3 réplicas vivas

PREGUNTA

17

Sin contestar

0.00 / 1.00

Un servicio de distribución de archivos observa que, mientras más usuarios se conectan, más capacidad total (ancho de banda y almacenamiento) tiene disponible el sistema. ¿Qué modelo arquitectónico explica mejor esta propiedad?

Seleccione una alternativa:

- ☐ a. Cliente-servidor puro, porque el servidor central crece proporcionalmente con la demanda.
- ☐ b. Cliente-servidor de tres capas, porque separa la lógica de negocio.
- ☐ c. Modelo híbrido, porque depende siempre de un servidor central para todas las operaciones.
- ☐ d. Peer-to-peer, porque cada nuevo nodo aporta recursos además de consumirlos.

La respuesta correcta es: Peer-to-peer, porque cada nuevo nodo aporta recursos además de consumirlos.

PREGUNTA

18

Sin contestar

0.00 / 1.00

Relacione: Situación de seguridad y propiedad que se protege

Relacione la opción correcta:

Antes de conceder el acceso, el sistema valida que el usuario es quien dice ser mediante un segundo factor.

Elegir...



El receptor recalcula un hash del mensaje y lo compara con el recibido para detectar si fue alterado en tránsito.

Elegir...



Cada orden de pago se firma digitalmente para que el emisor no pueda negar después haberla emitido.

Elegir...



Un usuario ya autenticado intenta abrir un módulo administrativo y el sistema se lo deniega según su rol.

Elegir...



La respuesta correcta es:

PREGUNTA

19

Sin contestar

0.00 / 1.00

¿Cuál afirmación distingue correctamente un sistema distribuido de un sistema paralelo de memoria compartida?.

Seleccione una alternativa:

- ☐ a. En el sistema distribuido no existe reloj global ni memoria compartida, por lo que la coordinación se basa en el paso de mensajes.
- ☐ b. El sistema paralelo no puede tener múltiples procesadores.
- ☐ c. El sistema distribuido siempre procesa más rápido.
- ☐ d. El sistema distribuido no admite fallos parciales.

La respuesta correcta es: En el sistema distribuido no existe reloj global ni memoria compartida, por lo que la coordinación se basa en el paso de mensajes.

PREGUNTA

20

Sin contestar

0.00 / 1.00

Un arquitecto debe decidir entre consistencia fuerte (mediante consenso Raft) o consistencia eventual para un contador de "me gusta" en una red social con millones de operaciones por segundo. ¿Cuál es la decisión más razonable y por qué?

Seleccione una alternativa:

- ☐ a. Consistencia fuerte, porque el consenso Raft no introduce latencia adicional, siendo indiferente, porque ambos modelos rinden igual bajo carga elevada
- ☐ b. Consistencia fuerte, porque un conteo exacto en tiempo real es crítico para la seguridad del sistema.
- ☐ c. Consistencia general, porque el dominio tolera imprecisiones generales y conviene priorizar disponibilidad y rendimiento bajo niveles de carga promedio, siendo aceptable que el valor converja después.
- ☐ d. Consistencia eventual, porque el dominio tolera imprecisiones temporales y conviene priorizar disponibilidad y rendimiento bajo alta carga, siendo aceptable que el valor converja después.

La respuesta correcta es: Consistencia eventual, porque el dominio tolera imprecisiones temporales y conviene priorizar disponibilidad y rendimiento bajo alta carga, siendo aceptable que el valor converja

después.

NAVEGACIÓN

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
17	18	19	20												

Correcto ☐ Parcial ☐ Incorrecto ☐